CSC309 A1 Writeup

Eric Gao and ShiRen Lu

March 2025

1 Model Design and Explanation

Our model is designed for a hotel and flight booking system. It includes models for users, hotels, rooms, flights, bookings, and notifications, along with additional supporting models. The core of the system is the **User** model, with all other models linked to it. The User model stores personal details, hotel ownership information, and booking records. In addition, we have included an ERD to help visualize the relationships between the models. Certain functionalities will be left to the frontend, for example turning the items in the cart into actual bookings. All the details required to generate an invoice exist in our current routes, the actual generation of the invoice will also be performed on the frontend. Our credit card verification doesn't actually charge anything, it just performs a few basic checks to see if the credit card is valid.

Below is an explanation of each model and their relationships.

1.1 User

Personal Details: Stores email, phone number, profile picture, and hashed password.

Hotel Ownership: Users can **own hotels**.

Bookings: Users can **book hotel rooms** and **flights**.

Cart: Contains flights and hotel bookings that the user has stored in their cart, these are stores in the temp relations

Notifications: Users receive system-generated notifications.

Relations: Each *User* can own multiple hotels, make multiple hotel and flight bookings, and receive notifications.

1.2 Hotel (Owned by a *User*)

Basic Info: Each hotel has a name, address, location, and star rating.

Ownership: A hotel is owned by a *User* (identified by 'ownerId').

Rooms: A hotel can have multiple rooms.

Images: A hotel can have multiple images.

Relations: Each *Hotel* belongs to a single *User* and can have multiple rooms and images.

1.3 Room (Belongs to a *Hotel*)

Unique Identifier: Each room has a unique ‘roomNumber‘ within its hotel.

Type: Rooms are linked to a **RoomType**, which defines pricing and amenities.

Bookings: A room can have multiple bookings over time.

Relations: Each *Room* belongs to a single *Hotel* and must have exactly one *RoomType*. Multiple bookings can be associated with a single room.

1.4 RoomType (Defines a category of *Rooms*)

Categories: Represents different room types (e.g., ”Deluxe Suite”, ”Standard Queen”).

Amenities: Stores amenities and ‘pricePerNight‘.

Key: Uses a composite key (‘hotelId‘, ‘typeName‘).

Relations: Each *RoomType* is linked to a single *Hotel* and can be associated with multiple rooms.

1.5 HotelBooking

Reservation Info: Stores user-room bookings with ‘checkInDate‘ and ‘checkOutDate‘.

Status: Tracks whether a booking is active or canceled.

Price: Stores the booking price.

Relations: Each *HotelBooking* is linked to a *User* and a *Room*.

1.6 FlightBooking

External Lookup: Stores only ‘bookingReference‘ and ‘lastName‘ for retrieval from AFS.

Relations: Each *User* can have multiple flight bookings.

1.7 FlightTemp

Cart Item: Stores the flight ID of a flight that the user may want to book later

1.8 HotelTemp

Cart Item: Stores the information a hotel booking the user may want to book later

1.9 Notifications

Message Delivery: Stores system-generated notifications for users.

Relations: Each *Notification* belongs to a single *User*.

2 Additional Minor Models

2.1 City

Geographical Data: Stores cities and their respective countries. Pulled from AFS during server startup.

2.2 Airport

Flight Infrastructure: Stores information about registered airports from AFS.

2.3 HotelImage

Hotel Media: Stores images associated with specific hotels.

2.4 RoomImage

Room Media: Stores images linked to room types.

2.5 ERD

Below is an included ERD to better help visualize our model: